



텍스트마이닝

강사 : 배현진

본 문서는 지정된 수취인만을 위해 작성되었으며, 중요한 정보나 지식재산권을 포함하고 있을 수 있습니다. 본 문서에 포함된 정보의 전부 또는 일부를 무단으로 제3자에게 공개, 배포, 복사 또는 사용하는 것은 엄격히 금지됩니다.
This document may contain confidential information and/or copyright material. This document is intended for the use of the addressee only. Any unauthorized dissemination, distribution, copying or use the information contained in this document is strictly prohibited.

텍스트 분석하기

텍스트마이닝이란?

비정형 데이터인 텍스트부터 인사이트 추출을 위해 데이터를 정제하고 범주화
여기서 정제와 범주화는 **텍스트의 패턴 및 특징**을 분석하는 것



텍스트 수집

크롤링 / 스크래핑



텍스트 정제

정규표현식,
텍스트 전처리...



텍스트 마이닝

빈도분석, 감성분석,
연관어분석, 토픽모델링...



인사이트 도출

데이터 시각화

텍스트 마이닝 VS 자연어처리

비슷하지만 조금 달라

데이터 분석중심!

텍스트마이닝

텍스트 데이터를 구조화

패턴 및 인사이트 발굴에 초점!

텍스트에서 어떻게하면 인사이트를 발굴할수 있을까?

주요 기술

- 감성분석
- 토픽모델링
- 문서클러스터링
- 연관분석

VS

자연스러운 언어표현 중심!

자연어처리

컴퓨터가 인간의 언어를 이해할 수 있도록 하는 것에 초점!

언어이해 및 생성에 초점

어떻게 컴퓨터가 언어를 우리가 '말'하는 의도대로

이해할 수 있게 만들 수 있을까?

주요 기술

- 형태소분석
- 기계 번역(ex, 구글번역기)
- 텍스트 변환 (TTS & STT)

텍스트마이닝 기초

: 형태소 분석 & 정규표현식

강사 : 배현진

본 문서는 지정된 수취인만을 위해 작성되었으며, 중요한 정보나 지식재산권을 포함하고 있을 수 있습니다. 본 문서에 포함된 정보의 전부 또는 일부를 무단으로 제3자에게 공개, 배포, 복사 또는 사용하는 것은 엄격히 금지됩니다.
This document may contain confidential information and/or copyright material. This document is intended for the use of the addressee only. Any unauthorized dissemination, distribution, copying or use the information contained in this document is strictly prohibited.

텍스트 전처리

그렇다면!

텍스트마이닝을 하는데 있어 컴퓨터가 잘~ 알아들을수 있도록 데이터를 올바르게 전처리 해주는것이 필요

1) 형태소 분석 : 원하는 품사 추출 / 불용어 처리...

오늘 나는 친구와 함께 영화를 볼 계획이다. ➡ 오늘 / 나 / 는 / 친구 / 와 / 함께 / 영화 / 를 / 볼 / 계획 / 이다 / .

오늘 나는 친구와 함께 영화를 볼 계획이다. ➡ ('오늘', 'Noun'), ('나', 'Noun'), ('는', 'Josa'), ('친구', 'Noun'), ('와', 'Josa'), ('함께', 'Adverb'), ('영화', 'Noun'), ('를', 'Josa'), ('볼', 'Noun'), ('계획', 'Noun'), ('이다', 'Josa'), ('.', 'Punctuation')

2) 정규표현식 : 원하는 텍스트 패턴 추출

제 전화번호는 010-4444-1111 이에요! ➡ ['010-4444-1111']

제 이메일은 aaaa@gmail.com 인데,
개인정보는 익명화해주세요 ➡ 제 이메일은 OOOOOOOOOO 인데,
개인정보는 익명화해주세요

텍스트 전처리 : 형태소 분석기 종류

◆ 영어 NLTK 라이브러리

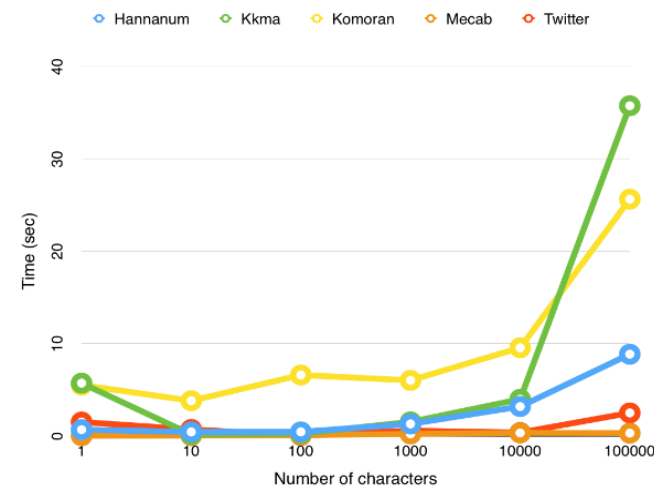
◆ 한국어 Konlpy 라이브러리

- Hannanum : 정제된 언어가 아니면 분석 품질 저하, 띄어쓰기 없는 문장 취약
- Okt : 어근화 가능, 다른 분석기에 비해 비정제 언어도 비교적 나쁘지 않은 편
- Komoran : 오타자 분석 잘되는 편 로딩시간이 길다, 띄어쓰기 없는 문장 취약
- Kkma : 문장이 늘어날수록 시간이 급격히 증가, 품사태깅이 구체적

*Mecab 속도 가장 빠름

+ 🚩 KiWi 등장

빠른 속도와 범용적인 성능을 지향 / 좋은점! 스페이스 보정 및 문장 분리 가능!



텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 영어 NLTK 라이브러리 : 설치 및 실행

설치코드

```
1 !pip install nltk
```

실행 코드

```
import nltk  
from nltk import pos_tag  
nltk.download("punkt")
```

텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 영어 형태소 분석 실행

예시 문장

'The little yellow dog barked at the Persian cat'

- 형태소 분석

```
1 nltk.word_tokenize(text)
```

```
['The', 'little', 'yellow', 'dog', 'barked', 'at', 'the', 'Persian', 'cat']
```

- 품사 태깅

- konlpy와 달리 명사만 추출가능한 함수는 존재하지않는다.
- tokenizer로 토큰화 된 리스트를 인풋데이터로 넣어주어야한다.

```
1 #형태소 단위로 분리
2 split_text = nltk.word_tokenize(text)
3 tagged_text = nltk.pos_tag(split_text)
```

결과

```
[('The', 'DT'),
 ('little', 'JJ'),
 ('yellow', 'JJ'),
 ('dog', 'NN'),
 ('barked', 'VBD'),
 ('at', 'IN'),
 ('the', 'DT'),
 ('Persian', 'JJ'),
 ('cat', 'NN')]
```


텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 영어 형태소 분석 실행

명사		형용사		동사	
NN	명사 단수	JJ	형용사	VB	동사 원형
NNS	명사 복수	JJR	형용사 비교급	VBD	동사 과거형
NNP	고유명사, 단수	JJS	형용사 최상급	VBG	동사 현재분사
NNPS	고유명사, 복수			VBN	동사 과거분사
				VBP	단수(3인칭X)
				VBZ	단수(3인칭O)

실습예제

✓ 명사만 추출하는 define 함수를 만들어보자

텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 영어 형태소 불용어

- nltk는 자주 사용하는 불용어 사전을 제공한다(리스트 형태로 제공).
- 불용어 사전은 모두 소문자이므로 형태소 분석시에 소문자로 변환하여 비교

```
1 nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to  
[nltk_data] C:\Users\hj\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping corpora\stopwords.zip.
```

```
True
```

```
1 stopwords = nltk.corpus.stopwords.words('english')
```

[불용어 사전]

텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 불용어 제거하기(소문자변환) : *문자열.lower()*

✓ 직접 문자열 넣기

```
1 'I LOVE YOU'.lower()  
'i love you'
```

✓ 변수를 지정하여 넣기

```
1 string = 'Are You Guys Enjoying The Class?'  
  
1 string.lower()  
'are you guys enjoying the class?'
```

텍스트 전처리 : 형태소 분석 (영어 NLTK)

◆ 불용어 제거하기

실습예제

명사		형용사		동사	
NN	명사 단수	JJ	형용사	VB	동사 원형
NNS	명사 복수	JJR	형용사 비교급	VBD	동사 과거형
NNP	고유명사, 단수	JJS	형용사 최상급	VBG	동사 현재분사
NNPS	고유명사, 복수			VBN	동사 과거분사
				VBP	단수(3인칭X)
				VBZ	단수(3인칭O)

- ✓ 불용어 사전이 있다고 가정했을 때 불용어를 제거하는 define 함수를 만들어보자
- ✓ 불용어가 제거되고, 형용사만 추출할 수 있는 define 함수를 만들어보자

텍스트 전처리 : 형태소 분석 (한국어 konlpy)

◆ Konlpy 설치하기

텍스트 전처리 : 형태소 분석 (한국어 konlpy)

◆ Konlpy 실행하기

실행 코드

```
from konlpy.tag import Hannanum
hannanum = Hannanum()
```

```
from konlpy.tag import Okt
okt = Okt()
```

```
from konlpy.tag import Komoran
komoran = Komoran()
```

```
from konlpy.tag import Kkma
kkma = Kkma()
```

- `okt.morphs(text)` : 형태소 분리
- `okt.nouns(text)` : 명사만 추출
- `okt.pos(text)` : 포스테깅

`norm`은 `normalize`의 약자로 문장을 정규화하는 역할
 - `stem`은 각 단어에서 어간을 추출하는 기능

- 문장 정규화 `okt.morphs( text, norm=True)`
- 어간 추출 `okt.morphs( text, stem=True)`

텍스트 전처리 : 형태소 분석 (한국어 konlpy)

◆ okt 만 제공하는 함수 * 비정형 텍스트 교정가능!

- 문장 정규화 `okt.morphs( text, norm=True)`
 `okt.pos( text, norm=True)`
- 어간 추출 `okt. morphs( text, stem=True)`
 `okt. pos( text, stem=True)`
- 동시사용 `okt.morphs( text, norm=True, stem=True)`
 `okt.pos( text, norm=True, stem=True)`

텍스트 전처리 : 형태소 분석 (한국어 konlpy)

◆ 불용어 제거하기

- 자주 사용되지만 특별한 의미 부여가 어려운 단어들을 제거
- 일반적으로 불용어 사전은 리스트 혹은 튜플 형태를 만들어 사용

실습예제

```
1 stopwords = ['있다', '되다', '하다']
```

✓ 불용어 사전이 있다고 가정했을 때 불용어를 제거하는 define 함수를 만들어보자

✓ 불용어가 제거되고, 형용사와 동사만 추출할 수 있는 define 함수를 만들어보자

→태그 이름 형용사 : 'Adjective', 동사 : 'Verb', 명사 : 'Noun'

텍스트 전처리 : 형태소 분석 (한국어 kiwi)

◆ Kiwi 설치 및 실행하기

설치코드

```
!pip install kiwipiepy
```

실행 코드

```
from kiwipiepy import Kiwi  
kiwi = Kiwi()
```

텍스트 전처리 : 형태소 분석 (한국어 kiwi)

◆ Kiwi 만 제공하는 함수

- 기본 불용어 사전 제공 Stopwords()
 - : 사전 추가 stopwords.add("재미")
 - : 사전 삭제 stopwords.remove("하하")
 - 띄어쓰기 교정 kiwi.space("나는텍스트마이닝을공부하고있어요")
 - 문장 분리 kiwi.split_into_sents("나는텍스트마이닝을공부하고있어요")
- kiwi.tokenize(text) : 포스테깅
= okt.pos(text)

문장 정규화 -> 기능 없음
okt.morphs(text, norm=True)

어간 추출 - > 기본적으로 키워는 지원
okt.morphs(text, stem=True)

텍스트 전처리 : 형태소 분석 (한국어 kiwi)

◆ Kiwi 태그 리스트

명사 (NN)

- NNG: 일반 명사 (예: "책", "컴퓨터")
- NNP: 고유 명사 (예: "서울")
- NNB: 의존 명사 (예: "에", "를")
- NNBC: 단위 명사 (예: "그램", "초")
- NR: 수사 (예: "둘", "백")

대명사 (NP)

- NP: 대명사 (예: "나", "너", "그것")
- NPS: 소유 대명사 (예: "내", "네")

형용사 (VA)

- VA: 형용사 (예: "크다", "예쁘다")
- VXV: 보조 형용사 (예: "지다", "같다")

부사 (MAG)

- MAG: 일반 부사 (예: "빠르게", "자주")
- MAJ: 접속 부사 (예: "그래서", "하지만")
- MAV: 보조 부사 (예: "아주", "좀")

동사 (VV)

- VV: 동사 (예: "먹다", "가다")
- VXA: 보조 동사 (예: "하다")

숫자 및 기타 (SF)

- SF: 기호
- SE: 접미사 (예: "들", "이")

정규표현식



텍스트 전처리 : 정규표현식

◆ 정규표현식 : Regular Expression(Regex)

텍스트 데이터의 패턴 발견

특정 문자열을 찾거나 바꾸는데 사용되는 라이브러리

(ex, 전화번호, 이메일...)

우리 가족의 전화번호를 알려줄게!

아버지의 전화번호는 010-1111-2222이고, 어머니의 전화번호는 010-3333-4444야

그리고 내 전화번호는 010-4444-5555야, 잘 저장해둬!



[010-1111-2222, 010-3333-4444, 010-4444-5555]

실행 코드

```
import re
```

텍스트 전처리 : 정규표현식

◆ 정규표현식 모듈:

모듈함수	의미
<code>re.compile("패턴")</code>	패턴을 컴파일 : 정규표현식을 정규식 객체로 변환 : 같은 패턴을 여러 번 사용할 때
<code>re.search("패턴 ", text)</code>	주어진 문자열 내에 첫번째로 일치하는 문자열
<code>re.findall("패턴 ", text)</code>	모든 일치 항목을 찾고 그룹 목록 반환(리스트형태) : 그룹단위 동작시 최소 단위부터 동작
<code>re.sub("패턴 ", "대체할 문자열", text)</code>	패턴에 필터링된 문자열을 대체하기

텍스트 전처리 : 정규표현식

◆ 자주 사용되는 문자열 :

자주 사용되는 문자열	의미	같은 의미
<code>\d</code>	숫자	<code>[0-9]</code>
<code>\D</code>	숫자가 아닌 문자	<code>[^0-9]</code>
<code>\s</code>	공백	
<code>\S</code>	공백이 아닌 문자	
<code>\w</code>	문자+숫자	<code>[a-zA-Z0-9]</code>
<code>\W</code>	문자+숫자가 아닌 것	<code>[^a-zA-Z0-9]</code>

텍스트 전처리 : 정규표현식

◆ 정규표현식 :

정규표현식에서는 특수 문자를 패턴에 넣을 수 없음
넣고 싶다면 특수문자 앞 \ 붙여주기

정규표현식	의미	패턴 예시	O	X
[]	문자한개	[abc]	a or b or c	abc(X)
[^]	Not을 의미	[^a]	abc → b , c	
.	임의의 문자 한 개(개행문자\n을 제외한 모든 문자)	a.c	abc	abb(X) a\nc(X)
+	앞의 문자가 1번이상 반복	a+b	aab(O) aaab(O)	
*	앞의 문자/숫자의 반복, 한번도 반복되지않을 수 있음	a*b	b(O) aab(O)	
{m,n}	앞의 문자가 최소 m, 최대 n의 반복	a{2,3}	aa(O) aaa(O)	aaaa(X)
{m}	앞의 문자의 m번의 반복	a{2}	aa(O)	aaa(X)
{m,}	앞의 문자가 m번 이상 반복	a{2,}	aa(O) aaa(O)	
()	문자를 묶는다. 여러 개의 문자로 대치	(ab)+	aaab(O) → ab abb(O) → ab	
?	앞의 문자/숫자가 없거나 한개	a?b	ab(O) b(O)	a(X) aab(X)

텍스트 전처리 : 정규표현식

◆ 많이 사용하는 정규표현식 패턴

- 전화번호
- 특수 기호 제거
- 한글자음 제거
- Email
- 알파벳제거
- HTML 태그제거
- 숫자제거
- URL

데이터 형태 그대로 저장하기 _ pickle

◆ pickle 라이브러리

데이터를 데이터프레임의 형태로 저장하기 곤란할 때
데이터를 타입 그대로 저장해주는 파이썬 내장 라이브러리

실행 코드

`import pickle`

✓ 저장하기

```
1 with open('save_data.pkl', "wb") as f:  
2     pickle.dump(data, f)
```

파일 이름.pkl

저장할 변수

✓ 불러오기

불러올 피클 파일 이름.pkl

```
1 with open('save_data.pkl', 'rb') as f:  
2     data = pickle.load(f)
```

피클 데이터의 변수이름 지정